浙江大学

《计算机组成与设计》
实验报告

实验名称 & 基于 RV32I 指令集的 RISC-V 微处理器设计

姓　　名 & _____

学　　号 & _____

院　　系 & _____

指导老师 & 屈民军唐奕

实验日期 & 2024 年 11 月 19 日

1 实验目的

1. 熟悉 RISC-V 指令系统。
2. 了解提高 CPU 性能的方法。
3. 掌握流水线 RISC-V 微处理器的工作原理。
4. 理解数据冒险、控制冒险的概念以及流水线冲突的解决方法。
5. 掌握流水线 RISC-V 微处理器的测试方法。
6. 了解软件实现数字系统的方法。

2 实验任务与要求

2.1 基本要求

设计一个流水线 RISC-V 微处理器。要求如下：

（1）至少运行下列 RV32I 核心指令。

算数运算指令：add、sub、addi

逻辑运算指令：and、or、xor、slt、sltu、andi、ori、xori、slti、sltiu

移位指令：sll、srl、sra、slli、srli、srai

条件分支指令：beq、bne、blt、bge、bltu、begu

无条件跳转指令：jal、jalr

数据传送指令：lw、sw、lui、auipc

空指令：nop

（2）采用 5 级流水线技术，对数据冒险实现转发或阻塞功能。

（3）在 Nexys Video 开发系统中实现 RISC-V 微处理器，要求 CPU 的运行速度大于 25MHz。

2.2 拓展要求

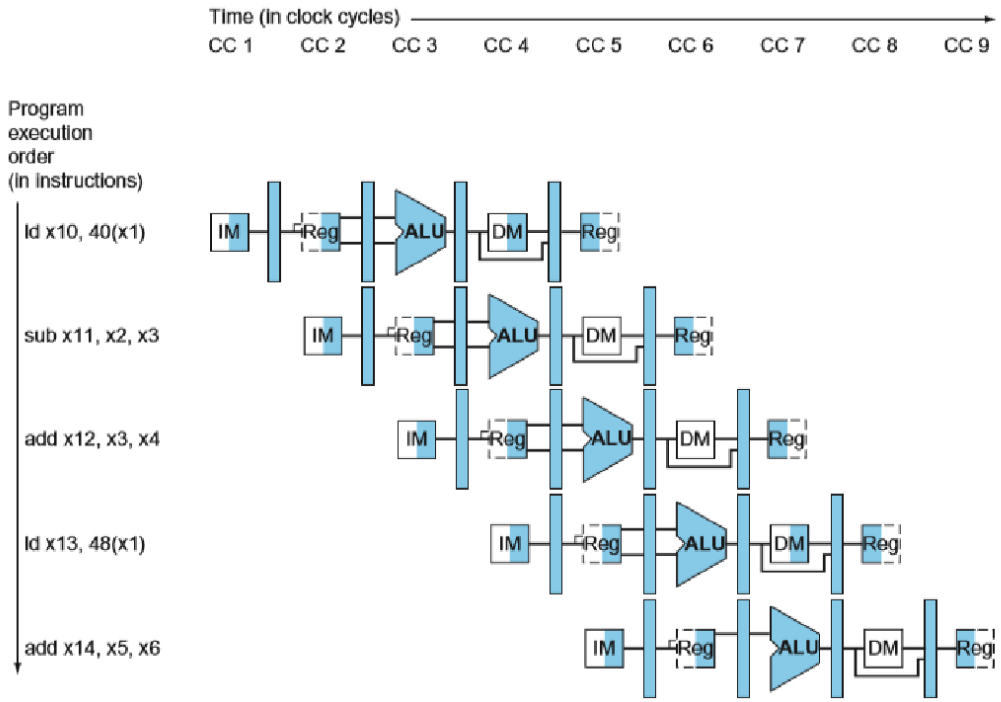
（1）要求设计的微处理器还能运行 lb、lh、ld、lbu、lhu、lwu、sb、sh 或 sd 等字节、半字和双字数据传送指令。

（2）要求设计的 CPU 增加异常（exception）、自陷（trap）、中断（interrupt）等处理方案。

3 具体设计

3.1 总体设计

流水线是数字系统中一种提高系统稳定性和工作速度的方法。根据 RISC-V 处理器指令的特点，将指令整体的处理过程分为取指令（IF）、指令译码（ID）、执行（EX）、存储器访问（MEM）和寄存器回写（WB）五级。一个指令的执行需要 5 个时钟周期，每个时钟上升沿来临时，此指令所代表的一系列数据和控制信息将转移到下一级处理。



流水线中的控制信号分为取指令（IF）、指令译码（ID）、执行（EX）、存储器访问（MEM）和寄存器回写（WB）五级，存在三类数据相关与数据转发，分别为一阶数据相关与转发（EX 冒险）、二阶数据相关与转发（MEM 冒险）、三阶数据相关（寄存器读写问题），这都是我们在设计流水线微处理器时需要考虑和处理的问题。具体设计下文将详细阐述。

3.2 指令译码模块（ID）的设计

指令译码模块的主要作用是从机器码中解析出指令，并根据解析结果输出各种控制信号。ID 模块主要由指令译码（Decode）、寄存器堆（Registers）、冒险检测、分支检测和加法器等组成。ID 模块的接口信息如表 30.3 所示。

（1）寄存器堆（Registers）子模块的设计

表 30.3 ID 模块的输入/输出引脚说明

引 脚名称	方向	说明
clk	Input	系统时钟
Instruction_id[31:0]		指令机器码
PC_id[31:0]		指令指针
RegWrite_wb		寄存器写允许信号，高电平有效
rdAddr_wb[4:0]		寄存器的写地址。
RegWriteData_wb[31:0]		写入寄存器的数据
MemRead_ex		冒险检测的输入
rdAddr_ex[4:0]		
MemtoReg_id	Output	决定回写的数据来源（0：ALU；1：存储器）
RegWrite_id		寄存器写允许信号，高电平有效
MemWrite_id		存储器写允许信号，高电平有效
MemRead_id		存储器读允许信号，高电平有效
ALUCode_id[3:0]		决定 ALU 采用何种运算
ALUSrcA_id		决定 ALU 的 A 操作数的来源（0：rs1；1：pc）
ALUSrcB_id[1:0]		决定 ALU 的 B 操作数的来源(2'b00：rs2；2'b01：imm；2'b10：常数 4)
Stall		ID/EX 寄存器清空信号，高电平表示插入一个流水线气泡
Branch		条件分支指令的判断结果，高电平有效
Jump		无条件分支指令的判断结果，高电平有效
IFWrite		阻塞流水线的信号，低电平有效
BranchAddr[31:0]		分支地址
Imm_id[31:0]		立即数
rdAddr_id[4:0]		回写寄存器地址
rs1Addr_id[4:0]		两个数据寄存器地址
rs2Addr_id[4:0]		
rs1Data_id[31:0]		寄存器两个端口输出数据
rs2Data_id[31:0]		

寄存器堆由 32 个 32 位寄存器组成，这些寄存器通过寄存器号进行读写存取。寄存器堆的原理框图如图 30.8 所示。因为读取寄存器不会更改其内容，故只需提供寄存号即可读出该寄存器内容。读取端口采用数据选择器即可实现读取功能。应注意的是，“0”号寄存器为常数 0。

对于往寄存器里写数据，需要目标寄存器号 (WriteRegister)、待写入数据 (WriteData)、写允许信号 (RegWrite) 三个变量。图 30.8 中 5 位二进制译码器完成地址译码，其输出控制目标寄存器的写使能信号 EN，决定将数据 WriteData 写入哪个寄存器。

对此，我们设计 Read Before Write 寄存器。

```
//Read before write Registers
//如果要写出 需要目标寄存器号，待写入数据，允许信号三个
//先分析指令，先读后写，读取写入
//后一层RAF用mux换数据

assign ReadData1= (ReadRegister1==5'b0)?32'b0:regs[ReadRegister1]; //rs1Data
assign ReadData2= (ReadRegister2==5'b0)?32'b0:regs[ReadRegister2]; //rs2Data
always @(posedge clk) begin
```

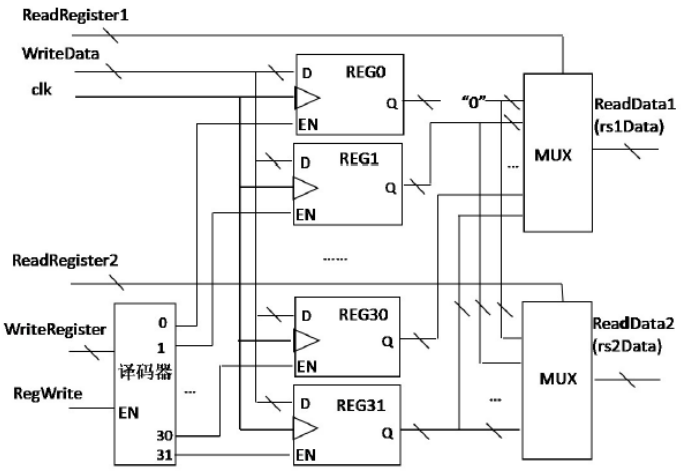


图 30.8　寄存器堆的原理框图

```
    if(RegWrite)
        regs[WriteRegister]<=WriteData; //数据写入
end
```

同时，在流水线型 CPU 设计中，寄存器堆设计还应解决三阶数据相关的数据转发问题。当满足三阶数据相关条件时，寄存器具有 Read After Write 特性。设计时，只需要在图 30.8 设计寄存器堆的基础上添加少量电路就可实现 Read After Write 特性，如图 30.9 所示。

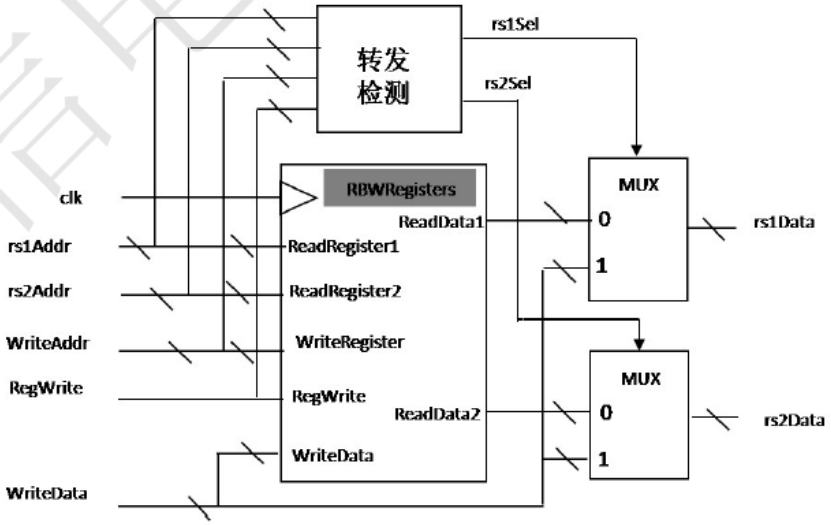


图 30.9 具有 Read After Write 特性寄存器堆的原理框图

对此，我们设计 Read After Write 寄存器堆。

```
//Read after write registers

```

```
//转发检测电路 30.1 30.2
//判断写入寄存器地址和读取寄存器地址是否一致，如一致，选择写入
assign rs1Sel= RegWrite && (WriteAddr!=0) && (WriteAddr==rs1Addr);
assign rs2Sel= RegWrite && (WriteAddr!=0) && (WriteAddr==rs2Addr);
//output
assign rs1Data= (rs1Sel==1'b0)?ReadData1:WriteData;
assign rs2Data= (rs2Sel==1'b0)?ReadData2:WriteData;
```

（2）指令译码（包含立即数产生电路）子模块的设计

该子模块主要作用是根据指令确定各个控制信号的值,同时产生立即数 Imm 和偏移量 offset。该模块是一个组合电路。

RISC-V 将指令分为 R、I、S、SB、U、UJ 等六类。因此，设置 R_type、I_type、SB_ type、LW、JALR、SW、LUI、AUIPC 和 JAL 等变量来表示指令类型，各变量的值由式 30.3 决定。

```
 // 设置R_type、I_type、SB_ type、LW、JALR、SW、LUI、AUIPC 和JAL 等变量来表示指令类型

 wire R_type, I_type, SB_type, LW, JALR, SW, LUI, AUIPC, JAL;
 assign R_type = (op == R_type_op);
 assign I_type = (op == I_type_op);
 assign SB_type = (op == SB_type_op);
 assign LW = (op == LW_op);
 assign JALR = (op == JALR_op);
 assign SW = (op == SW_op);
 assign LUI = (op == LUI_op);
 assign AUIPC = (op == AUIPC_op);
 assign JAL = (op == JAL_op);
```

A 只有 LW 指令读取存储器且回写数据取自存储器，所以有

```
assign MemtoReg = LW;
assign MemRead = LW;
```

B 只有 SW 指令会对存储器写数据，所以有

```
assign MemWrite = SW;
```

C 需要进行回写的指令类型有 R_type、I_type、LW、JALR、LUI、AUIPC 和 JAL。所以有

```
assign RegWrite = R_type || I_type || LW || JALR || LUI || AUIPC || JAL;
```

D 只有 JALR 和 JAL 两条无条件分支指令，所以有

```
assign Jump = JALR || JAL;
```

E 操作数 A 和 B 的选择信号的确定可见表 30.4

表 30.4　操作数选择信号的功能表

类型	ALUSrcA_id	ALUSrcB_id[1:0]	说明
R_type	0	2'b00	rd=rs1　op　rs2
I_type	0	2'b 01	rd=rs1　op　imm
LW	0	2'b 01	rs1　+　imm
SW	0	2'b 01	rs1　+　imm
JALR	1	2'b 10	rd=pc　+　4
JAL	1	2'b 10	rd=pc　+　4
LUI	1'bx	2'b 01	rd=　　imm
AUIPC	1	2'b 01	rd=pc　+　imm

```
assign ALUSrcA= (LUI)?1'bx:(JALR || JAL || AUIPC);
assign ALUSrcB[1]= JAL || JALR;
assign ALUSrcB[0]= ~(R_type || JAL || JALR);
```

F ALUCode 的确定

除了条件分支指令，其它指令都需要 ALU 执行运算，共有 11 种不同运算，ALUCode 信号需用 4 位二进制表示。最主要为加法运算，设为默认算法，ALUCode 的功能表如表 30.5 所示。注意：表中 funct7[6] 与 funct6[5] 在指令中为同一位置，即 instruction[30]。

G 立即数产生电路（ImmGen）设计

I_type、SB_type、LW、JALR、SW、LUI、AUIPC 和 JAL 这几类指令均用到立即数。由于 I_type 的算术逻辑运算与移位运算指令的立即数构成方法不同，这里再设定一个变量 Shift 来区分两者。Shift=1 表示移位运算，否为则算术逻辑运算。Shift 值由式 (30.11) 计算。

```
Shift=（funct3==1）||（funct3==5）　(30.11)
```

立即数构成和扩展方法如表 30.6 所示，表中的 inst 即 instruction。

因为立即数产生电路比较简单，所以直接在 I D 顶层模块设计中即可实现。

（3）分支检测（Branch Test）电路的设计

分支检测电路主要用于判断分支条件是否成立,在 Verilog HDL 可以用比较运算符号“>”、“==”和“<”描述，但要注意符号数和无符号数的处理方法不同。在这里，我们用加法器来实现。

表 30.6　立即数产生方法

类别	Shift	Imm	offset
I_type	1	{26'd0,inst[25:20]}	-
I_type	0	{{20{inst[31]}},inst[31:20]}	-
LW	x		-
JALR	x	-	{{20{inst[31]}},inst[31:20]}
SW	x	{{20{inst[31]}},inst[31:25],inst[11:7]}	-
JAL	x	-	{{11{inst[31]}}, inst[31], inst[19:12] , inst[20], inst[30:21], 1'b0}
LUI	x	{inst[31:12], 12'd0}	-
AUIPC	x		-
SB_type	x	-	{{19{inst[31]}}, inst[31], inst[7],inst[30:25] , inst[11:8], 1'b0}

1、用一个 32 位加法器完成 rs1Data+（ rs2Data）+1（即 rs1Data-rs2Data），设结果为 sum [31:0] 。备注：此处使用 lab8 中编写的 32 位进位选择加法器。

2、确定比较运算的结果。对于比较运算来说，如果最高位不同，即 rs1Data [31]　rs2Data [31]，可根据 rs1Data [31]、rs2Data [31] 决定比较结果，但是应注意符号数、无符号数的最高位 rs1Data [31]、rs2Data [31] 代表意义不同。若两数最高位相同，则两数之差不会溢出，所以比较运算结果可由两个操作数之差的符号位 sum[31] 决定。

在【符号数】比较运算中，rs1Data<rs2Data 有以下两种情况：

当 rs1Data 为负数、rs2Data 为 0 或正数: rs1Data[31]&&(rs2Data[31])

当 rs1Data、rs2Data 符号相同，sum 为负: (rs1Data[31] rs2Data[31]) &&| sum[31]

因此，符号数 rs1Data<rs2Data 比较运算结果为:

isLT=rs1Data[31]&&(rs2Data[31]) Il (rs1Data[31] rs2Data[31])&& sum[31]

同样地，【无符号数】比较运算中，rs1Data<rs2Data 有以下两种情况：

当 rs1Data 最高位为 0、rs2Data 最高位为 1：(rs1Data[31])&& rs2Data[31]

当 rs1Data、rs2Data 最高位相同，sum 为负：(rs1Data[31] rs2Data[31]) && sum[31]

因此，无符号数比较运算结果为:

isLTU= (rs1Data[31])&& rs2Data[31] || (rs1Data[31]　rs2Data[31]) && sum[31]

最后用数据选择器完成式 (30.14) 即可完成分支检测。

$$\text{Branch} = \begin{cases} \sim(|\,\text{sum}[31:0]); & \text{SB_type} \ \&\&\ (\text{funct3} == \text{beq_funct3}) \\ |\,\text{sum}[31:0]; & \text{SB_type} \ \&\&\ (\text{funct3} = \text{bne_funct3}) \\ \text{isLT}; & \text{SB_type} \ \&\&(\text{funct3} = \text{blt_funct3}) \\ \sim \text{isLT}; & \text{SB_type} \ \&\&\ (\text{funct3} = \text{bge_funct3}) \\ \text{isLTU}; & \text{SB_type} \ \&\&\ (\text{funct3} = \text{bltu_funct 3}) \\ \sim \text{is}LTU; & \text{SB_type} \ \&\&\ (\text{funct3} = \text{bgeu_funct 3}) \\ 0 & \text{others} \end{cases} \tag{30.14}$$

代码实现如下：

```

//分支检测 表30.14
always @(*) begin
    if(SB_type) begin
        case(func3)
            beq_func3: Branch= ~(|sum[31:0]);
            bne_func3: Branch= |sum[31:0];
            blt_func3: Branch= isLT;
            bge_func3: Branch= ~isLT;
            bltu_func3: Branch= isLTU;
            bgeu_func3: Branch= ~isLTU;
            default: Branch= 0;
        endcase
    end
    else
        Branch= 1'b0;
end
endmodule
```

(4) 冒险检测功能电路（Hazard Detector）的设计：

冒险成立的条件为：

A 上一条指令必须是 lw 指令（MemRead_ex=1）；

B 两条指令读写同一个寄存器（rdAddr_ex=rs1Addr_id 或 rdAddr_ex=rs2Addr_id）。当冒险成立应清空 ID/EX 寄存器并且阻塞流水线 ID 级、IF 级流水线，代码设计如下：

```
    // 冒险检测
assign Stall = ((rdAddr_ex==rs1Addr_id) || (rdAddr_ex==rs2Addr_id)) &&
    MemRead_ex;
assign IFWrite = ~Stall; //低电平有效,所以高电平正常
```

3.3 执行模块（EX）的设计

执行模块主要由 ALU 子模块、数据前推电路（Forwarding）及若干数据选择器组成。执行模块的接口信息如表 30. 7 所示。

表 30.7　EX 模块的输入/输出引脚说明

引脚名称	方向	说明
ALUCode_ex[3:0]	Input	决定 ALU 采用何种运算
ALUSrcA_ex		决定 ALU 的 A 操作数的来源（rs1、PC）
ALUSrcB_ex[1:0]		决定 ALU 的 B 操作数的来源(rs2、imm 和常数 4)
Imm_ex[31:0]		立即数
rs1Addr_ex[4:0]		rs1 寄存器地址
rs2Addr_ex[4:0]		rs2 寄存器地址
rs1Data_ex[31:0]		rs1 寄存器数据
rs2Data_ex[31:0]		rs2 寄存器数据
PC_ex[31:0]		指令指针
RegWriteData_wb[31:0]		写入寄存器的数据
ALUResult_mem[31:0]		ALU 输出数据
rdAddr_mem[4:0]		寄存器的写地址
rdAddr_wb[4:0]		
RegWrite_mem		寄存器写允许信号
RegWrite_wb		
ALUResult_ex[31:0]	Output	ALU 运算结果
MemWriteData_ex[31:0]		存储器的回写数据
ALU_A [31:0]		ALU 操作数，测试时使用
ALU_B [31:0]		

（1）ALU 子模块的设计

算术逻辑运算单元（ALU）提供 CPU 的基本运算能力，如加、减、与、或、比较、移位等。具体而言，ALU 输入为两个操作数 A、B 和控制信号 ALUCode，由控制信号 ALUCode 决定采用何种运算，运算结果为 ALUResult。整理表 30.5 所示的 ALUCode 的功能表，可得到 ALU 的功能表，如表 30.8 所示。

表 30.8　ALU 的功能表

ALUCode	ALUResult
4’b0000	A + B
4’b0001	A-B
4’b0010	B
4’b0011	A&B
4’b0100	A ^ B
4’b0101	A | B
4’b0110	A << B
4’b0111	A >> B
4’b1000	A>>>B
4’b1001	A<B? 1:0，其中 A、B 为有符号数
4’b1010	A<B? 1:0，其中 A、B 为无符号数

如表 30.8 所示，ALU 需执行多种运算，为了提高运算速度，本设计可同时进行各种运算，

再根据 ALUCode 信号选出所需结果。

ALU 的基本结构如图 30.10 所示。可以看出，我们需要分别进行加减电路、比较电路、算数右移运算电路的设计考虑。

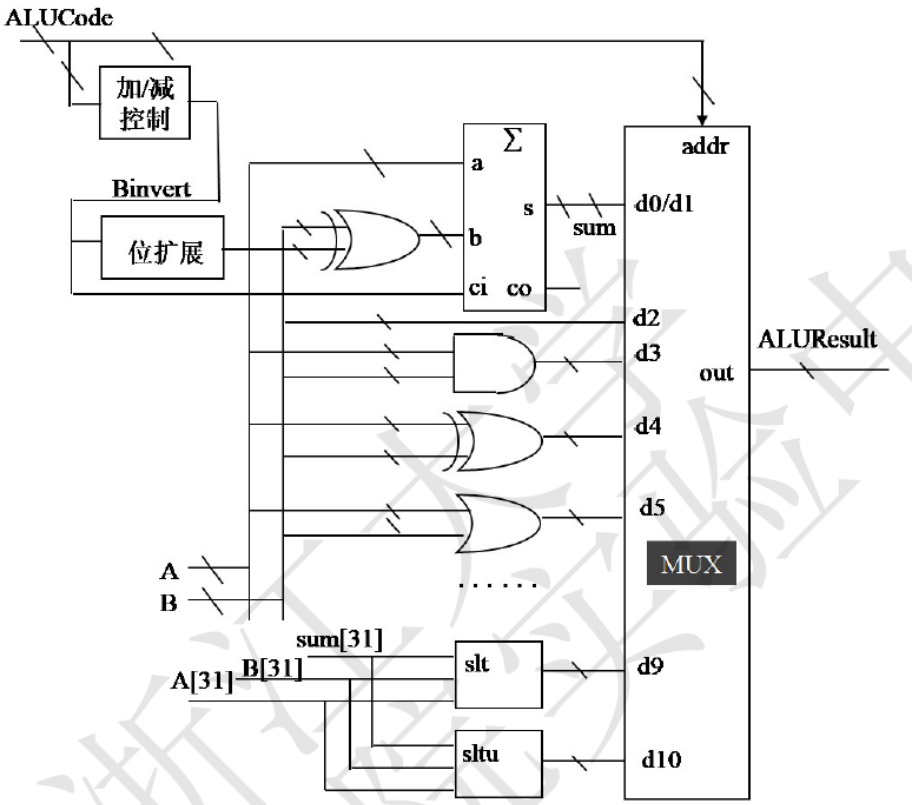


图 30.10　ALU 结构框图

（2）数据前推电路的设计

操作数 A 和 B 分别由数据选择器决定，数据选择器地址信号 ForwardA、ForwardB 的含义如表 30.9 所示。

表 30.9　前推电路输出信号的含义

地　　址	操作数来源	说　　明
ForwardA= 2’b00	rs1Data_ex	操作数 A 来自寄存器堆
ForwardA=2’b01	RegWriteData_wb	操作数 A 来自二阶数据相关的转发数据
ForwardA= 2’b10	ALUResult_mem	操作数 A 来自一阶数据相关的转发数据
ForwardB= 2’b00	rs2Data_ex	操作数 B 来自寄存器堆
ForwardB= 2’b01	RegWriteData_wb	操作数 B 来自二阶数据相关的转发数据
ForwardB=2’b10	ALUResult_mem	操作数 B 来自一阶数据相关的转发数据

结合一阶、二阶数据冒险相关判断条件，设计如下：

```
    assign ForwardA[0]= RegWrite_wb && (rdAddr_wb!=0) && (rdAddr_mem!=rs1Addr_ex)
        && (rdAddr_wb==rs1Addr_ex);
    assign ForwardA[1]= RegWrite_mem && (rdAddr_mem!=0) && (rdAddr_mem==
       rs1Addr_ex);
    assign ForwardB[0]= RegWrite_wb && (rdAddr_wb!=0) && (rdAddr_mem!=rs2Addr_ex)
       && (rdAddr_wb==rs2Addr_ex);
    assign ForwardB[1]= RegWrite_mem && (rdAddr_mem!=0) && (rdAddr_mem==
       rs2Addr_ex);
```

（3）EX 顶层模块的设计

EX 顶层模块中，除了 ALU 和 Forwarding 子模块，还有一个对 ALU 的两操作数进行选择的选择器电路。

```
begin
    case (ForwardA)
        2'b00: A_0 = rs1Data_ex;
        2'b01: A_0 = RegWriteData_wb;
        2'b10: A_0 = ALUResult_mem;
    endcase
    case (ForwardB)
        2'b00: B_0 = rs2Data_ex;
        2'b01: B_0 = RegWriteData_wb;
        2'b10: B_0 = ALUResult_mem;
    endcase
]
```

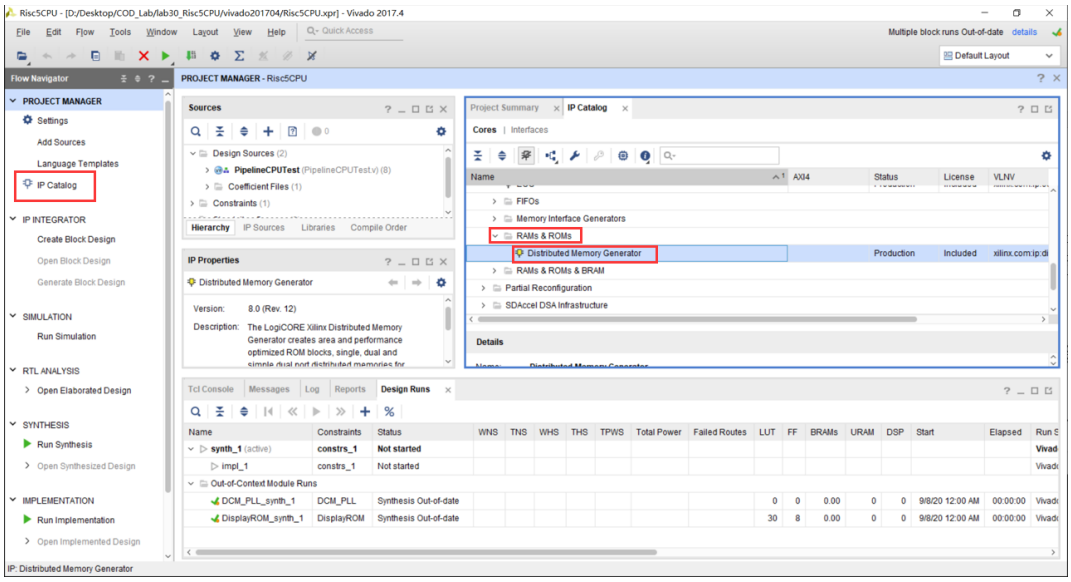
3.4 数据存储器模块 (DataRAM) 的设计

数据存储器可用 Xilinx 的 IP 内核实现。考虑到 FPGA 的资源，数据存储器可设计为容量为 64x32bit 的单端口 RAM，输出采用组合输出 (Non Registered)。

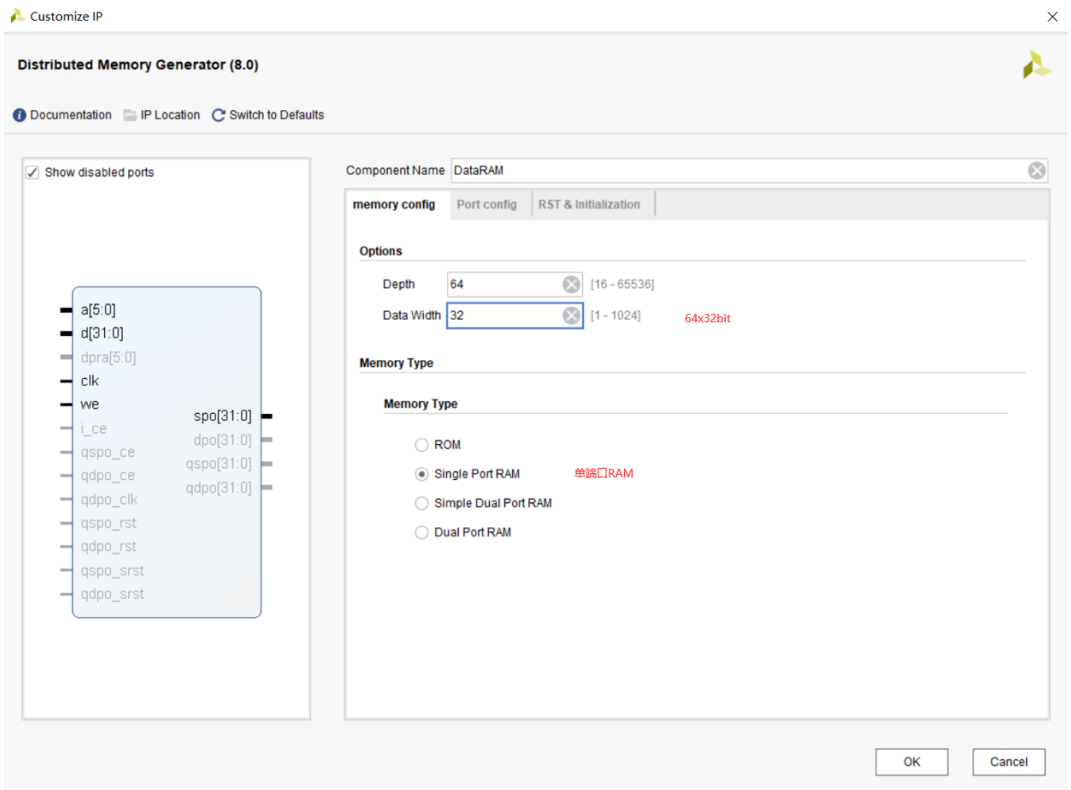
由于数据存储器容量为 64x32bit, 故存储器地址共 6 位，与 ALUResult_mem[7:2] 连接。

具体操作步骤如下：

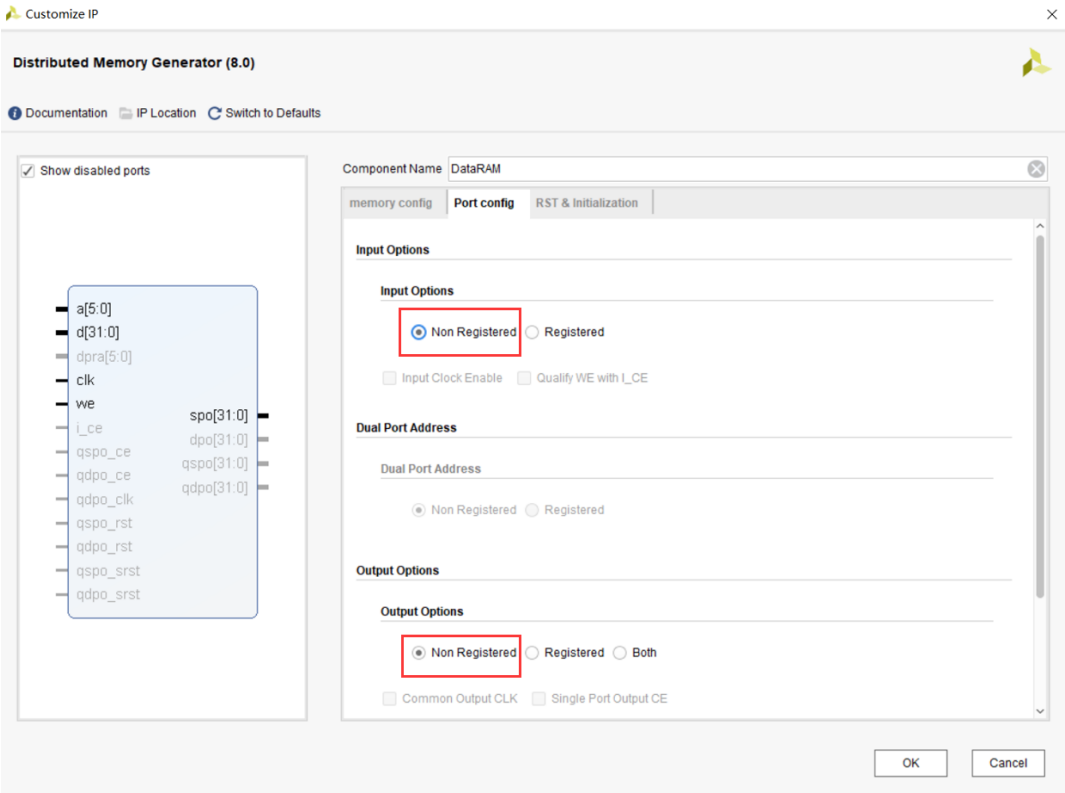
（1）打开 PROJECT MANAGER 栏中的 IP Catalog，在 RAMs & ROMs 中选择 Distributed Memory Generator，进入配置界面。



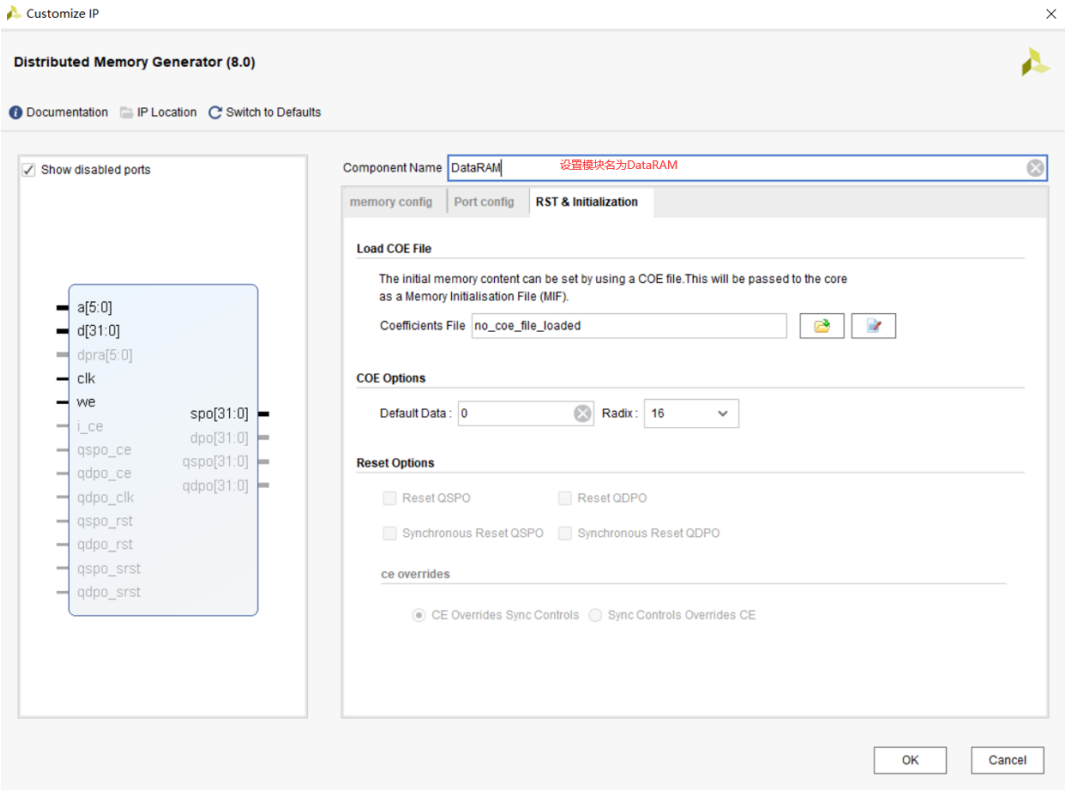
（2）在第一列 Memory config 中进行如下配置：Options 中深度（Depth）设置为 64，Data Width 为 32 位。Memory Type 中选择 Single Port RAM。



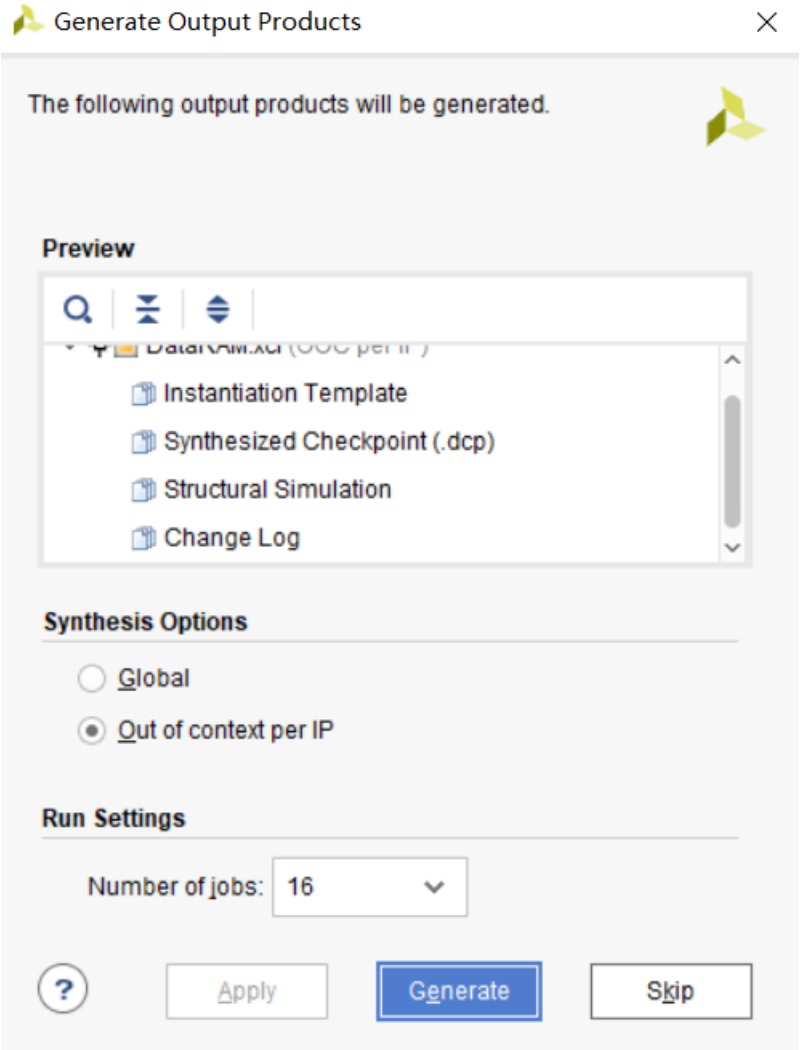
（3）在第二列 Port Config 中进行如下配置：Input Options 和 Output Options 中选择 Non Registered。


Customize IP
Distributed Memory Generator (8.0)
Documentation
IP Location
Switch to Defaults
Show disabled ports
a[5:0]
d[31:0]
dpra[5:0]
clk
we
i_ce
qspo_ce
qdpo_ce
qdpo_clk
qspo_rst
qdpo_rst
qspo_srst
qdpo_srst
spo[31:0]
dpo[31:0]
qspo[31:0]
qdpo[31:0]
Component Name
DataRAM
memory config
Port config
RST & Initialization
Input Options
Input Options
Non Registered
Registered
Input Clock Enable
Qualify WE with I_CE
Dual Port Address
Dual Port Address
Non Registered
Registered
Output Options
Output Options
Non Registered
Registered
Both
Common Output CLK
Single Port Output CE
OK
Cancel

（4）设置模块名。


Customize IP
Distributed Memory Generator (8.0)
Documentation
IP Location
Switch to Defaults
Show disabled ports
a[5:0]
d[31:0]
dpra[5:0]
clk
we
i_ce
qspo_ce
qdpo_ce
qdpo_clk
qspo_rst
qdpo_rst
qspo_srst
qdpo_srst
spo[31:0]
dpo[31:0]
qspo[31:0]
qdpo[31:0]
Component Name
DataRAM
设置模块名为DataRAM
memory config
Port config
RST & Initialization
Load COE File
The initial memory content can be set by using a COE file.This will be passed to the core
as a Memory Initialisation File (MIF).
Coefficients File
no_coe_file_loaded
COE Options
Default Data :
0
Radix :
16
Reset Options
Reset QSPO
Reset QDPO
Synchronous Reset QSPO
Synchronous Reset QDPO
ce overrides
CE Overrides Sync Controls
Sync Controls Overrides CE
OK
Cancel

（5）选择 Out of context per IP，点击 Generate。



完成以上设置后，会自动生成新的 IP 核相关文件，位置为... vivado201704 Risc5CPU.srcs sources_1 DataRAM（这个为你为 IP 核设置的名字），含有 DataRAM_sim_netlist.v 文件，sim 文件夹中有 DataRAM.v 文件，simulation 文件夹中有 dist_mem_gen_v8_0.v 文件，这三个文件将在后面仿真用到，在此提前说明。

3.5 取指令级模块 (IF) 的设计

IF 模块由指令指针寄存器 (PC)、指令存储器子模块 (Instruction ROM)、指令指针选择器 (MUX) 和一个 32 位加法器组成，IF 模块接口信息如表 30.10 所示。

表 30.10　IF 模块的输入/输出引脚说明

引脚名称	方向	说明
clk	Input	系统时钟
reset		系统复位信号，高电平有效
Branch		条件分支指令的条件判断结果
Jump		无条件分支指令的条件判断结果
IFWrite		流水线阻塞信号
JumpAddr[31:0]		分支地址
Instruction [31:0]	Output	指令机器码
IF_flush		流水线清空信号
PC [31:0]		PC 值

对于 IF 模块的设计，由数据通路相关知识，如果不发生跳转（无 Branch 或 Jump 信号），则直接从当前 PC 所指向的内存地址取指令，完成取指令操作后，PC+4；如果发生跳转，则需先修改 PC 值为 PC = JumpAddr，再从 PC 所指向的内存地址取指令，完成取指令操作后，PC+4，同时因为跳转的发生还需要清空当前流水线（因为无跳转时顺序取指令执行，跳转的发生使得预先取得的指令失效）；此外，如果发生阻塞（IFWrite=0），该时钟周期 PC 值需要保持不变。

其中 InstructionROM 为一个指令存储器，存放一段简单测试程序的机器码，实验已给出，无需设计。

3.6　流水线寄存器的设计

对于整个 CPU 的设计，我们还需要流水线寄存器，它负责将流水线的各部分分开，共有 IF/ID、ID/EX、EX/MEM、MEM/WB 四组，对四组流水线寄存器要求不完全相同，因此设计也有不同考虑。

对于 EX/MEM、MEM/WB 两组流水线寄存器只是普通的 D 型寄存器。

ID/EX、IF/ID 流水线寄存器需要有一些特别的设计。

当流水线发生数据冒险时，需要清空 ID/EX 流水线寄存器而插入一个气泡，因此 ID/EX 流水线寄存器是一个带同步清零功能的 D 型寄存器。

当流水线发生数据冒险时，需要阻塞 IF/ID 流水线寄存器；若跳转指令或分支成立，则还需要清空 ID/EX 流水线寄存器。因此，IF/ID 流水线寄存器除同步清零功能外，还需要具有保持功能（即具有使能 EN 信号输入）。

```
    if(reset|IF_flush)
      begin
          PC_id <= 0;
          Instruction_id <= 0;
      end
```

```
        else if(ifwrite)
        begin
            PC_id <= PC_if;
            Instruction_id <= Instruction_if;
        end
        else
        //如果要写入，插入气泡
        begin
            PC_id <= PC_id;
            Instruction_id <= Instruction_id;
        end
```

3.7 顶层设计

这部分设计按照图 30.3 所示的原理框图连接各模块即可。为了测试方便，可将关键变量输出，关键变量有: 指令指针 PC、指令码 Instruction_ id. 流水线插入气泡标志 Stall、分支标志 JumpFlag 即 Jump, Branch、ALU 输入输出 (ALU_A、ALU_B、ALUResult_ex) 和数据存储器的输出 MemDout_mem，由于该部分代码较长，且只是模块间的连线，并没有特别，故不附到报告中。

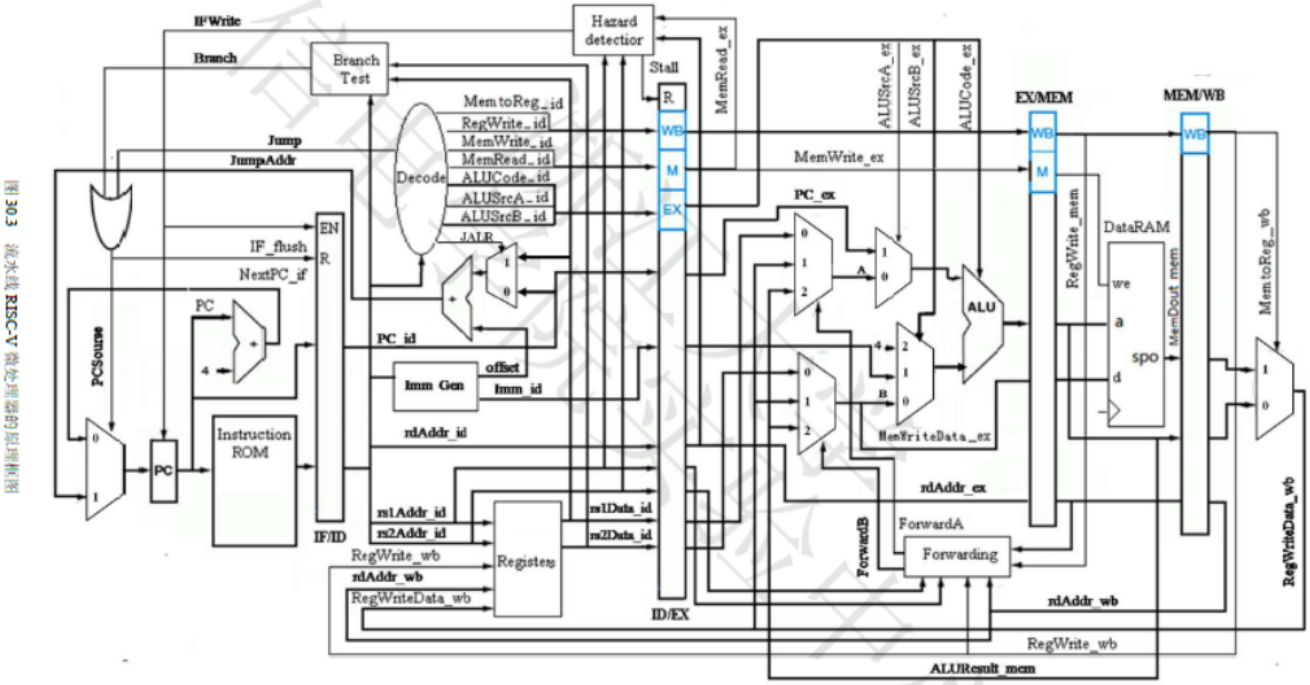
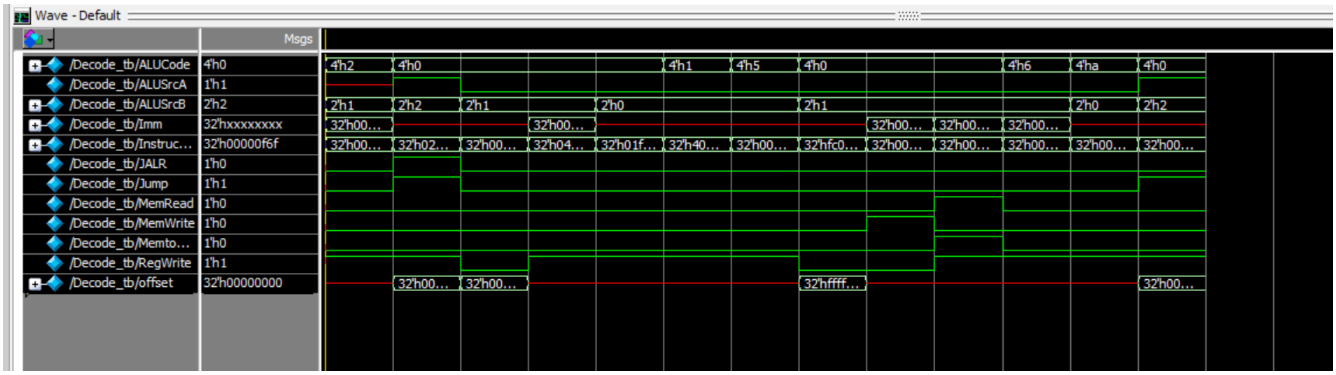


图 30.3 流水线 RISC-V 微处理器的原理框图

4 实验结果

4.1 Decode 模块仿真结果及分析



lui x30, 0x3000，为 U 型，输出应为 Imm = 0x0000_3000，offset 无意义，ALUCode=4’d2，ALUSrcA=0，ALUSrcB=2’b01，LUI 指令需要回写，RegWrite=1，波形显示正确。

jalr X31, later(X0)，为 I 型，输出应为 Imm 无意义，offset=0x0000_0020，ALUCode=4’d0，ALUSrcA=1，ALUSrcB=2’b10，JALR=1，Jump=1，JALR 指令需要回写，RegWrite=1，波形显示正确。

bne X0, X0, end，为 SB 型，输出应为 Imm 无意义，offset=0x0000_0018，ALUCode=4’d0，ALUSrcA=0，ALUSrcB=2’b01，BNE 指令不需要回写，RegWrite=0，波形显示正确。

addi X5, X30, 42，为 I 型，输出应为 Imm =0x0000_0042，offset 无意义，ALUCode=4’d0，ALUSrcA=0，ALUSrcB=2’b01，ADDI 指令需要回写，RegWrite=1，波形显示正确。

add X6, X0, X31，为 R 型，输出应为 Imm 无意义，offset 无意义，ALUCode=4’d0，ALUSrcA=0，ALUSrcB=2’b00，ADD 指令需要回写，RegWrite=1，波形显示正确。

sub X7, X5, X6，为 R 型，输出应为 Imm 无意义，offset 无意义，ALUCode=4’d1，ALUSrcA=0，ALUSrcB=2’b00，SUB 指令需要回写，RegWrite=1，波形显示正确。

or X28, X7, X5，为 R 型，输出应为 Imm 无意义，offset 无意义，ALUCode=4’d5，ALUSrcA=0，ALUSrcB=2’b00，OR 指令需要回写，RegWrite=1，波形显示正确。

beq X0, X0, earlier，为 SB 型，输出应为 Imm 无意义，offset=0xffff_ffd4，ALUCode=4’d0，ALUSrcA=0，ALUSrcB=2’b01，BEQ 指令不需要回写，RegWrite=0，波形显示正确。

sw X28, 0C(X0)，为 S 型，输出应为 Imm =0x0000_000c，offset 无意义，ALUCode=4’d0，ALUSrcA=0，ALUSrcB=2’b01，MemWrite=1，波形显示正确。

lw X29, 04(X6)，为 I 型，输出应为 Imm =0x0000_0004，offset 无意义，ALUCode=4’d0，ALUSrcA=0，ALUSrcB=2’b01，MemRead=1，MemtoReg=1，RegWrite=1，波形显示正确。

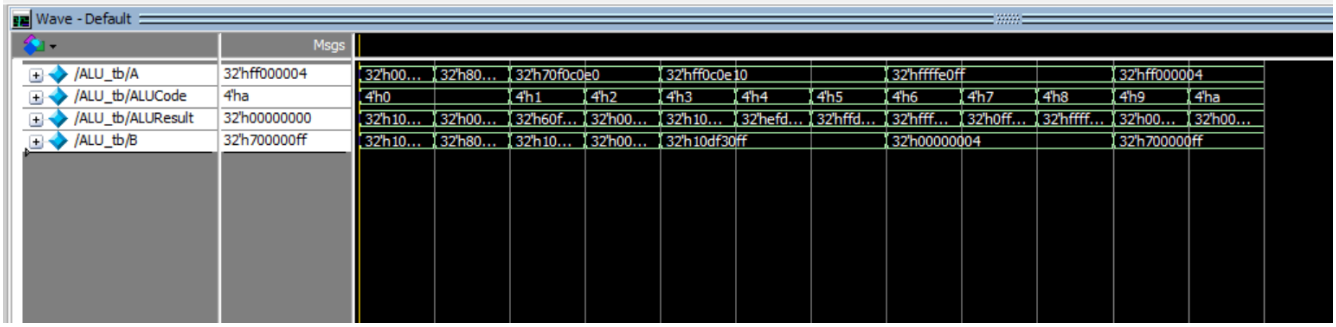
sll X5, X29,2，为 I 型，输出应为 Imm =0x0000_0002，offset 无意义，ALUCode=4’d6，ALUSrcA=0，ALUSrcB=2’b01，RegWrite=1，波形显示正确。

sltu X28, X6,X7，为 R 型，输出应为 Imm 无意义，offset 无意义，ALUCode=4’d10，ALUSrcA=0，ALUSrcB=2’b00，RegWrite=1，波形显示正确。

jal X31, done，为 UJ 型，输出应为 Imm 无意义，offset=0x0000_0000，ALUCode=4’d1，ALUSrcA=1，ALUSrcB=2’b10，SUB 指令需要回写，RegWrite=1，波形显示正确。

对照表 30.4 操作数选择信号功能表、表 30.5 ALUCode 功能表、表 30.6 立即数产生方法检查各项输出，均无误,Decode 模块功能实现。

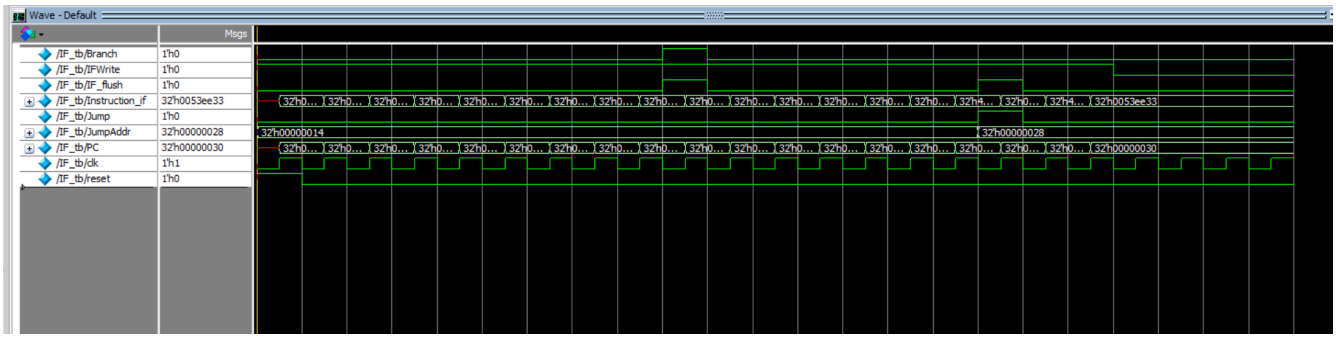
4.2 ALU 模块仿真结果及分析



具体输出结果看下表，输入为 A 和 B 的值，根据 ALUcode 的数值选择逻辑运算方式，得到 ALUResult，结果均正确，ALU 模块功能实现。

A	B	ALUcode	ALUResult
32’h00004012	32’h1000200f	0(+)	32’h10006021
32’h80000000	32’h80000000	0(+)	32’h00000000
32’h70f0c0e0	32’h10003054	1(-)	32’h60f0908c
32’h70f0c0e0	32’h00003000	2(B)	32’h00003000
32’hff0c0e10	32’h10df30ff	3(&)	32’h100c0010
32’hff0c0e10	32’h10df30ff	4(^)	32’hefd33eef
32’hff0c0e10	32’h10df30ff	5(|)	32’hffdf3eff
32’hffffe0ff	32’h00000004	6(«)	32’hfffe0ff0
32’hffffe0ff	32’h00000004	7(»)	32’h0ffffe0f
32’hffffe0ff (保持符号位不变)	32’h00000004	8(»>)(算数右移)	32’hfffffe0f
32’hff00000004 (此时 A 为负数，B 为正数)	32’h700000ff	9(A<B?1:0)(有符号数)	32’h00000001
32’hff00000004	32’h700000ff	10(A<B?1:0) (无符号数)	32’h00000000

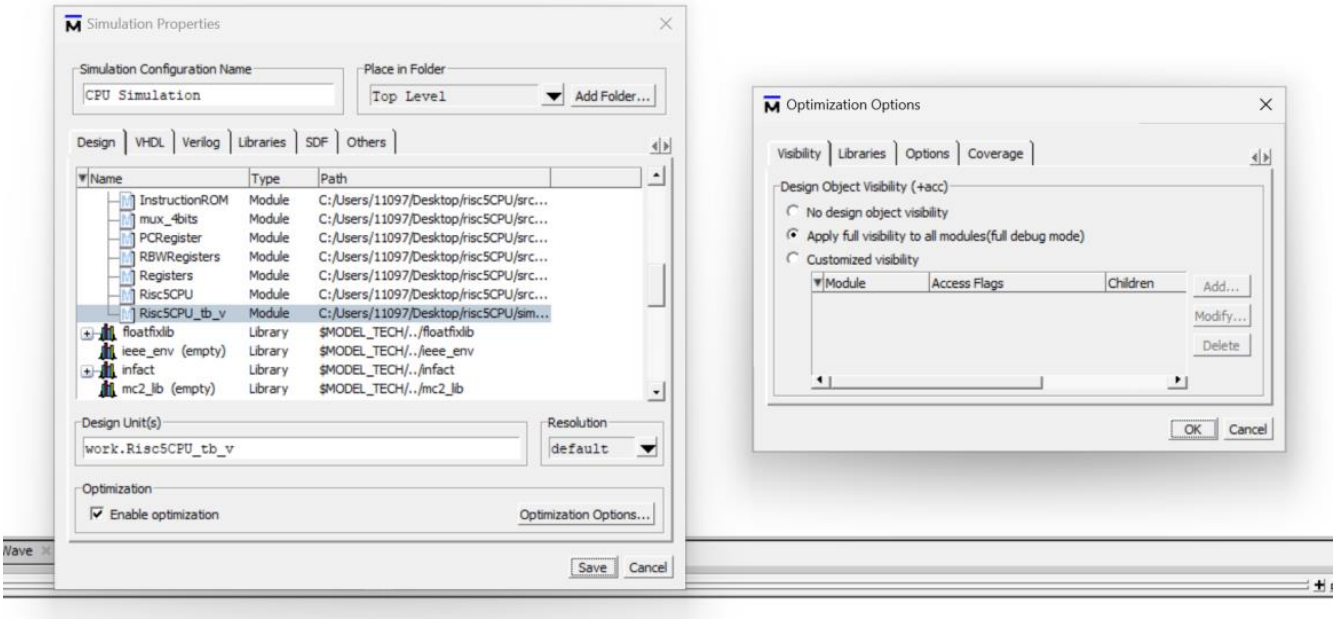
4.3 IF 模块仿真结果及分析



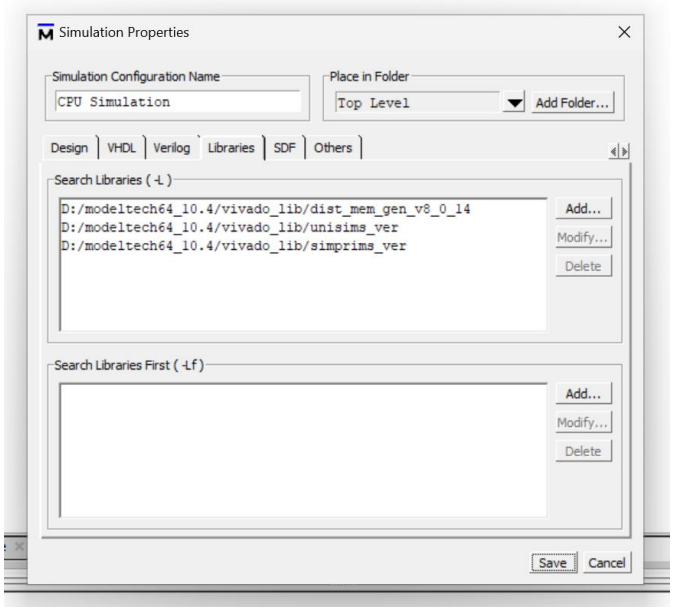
可以看出，PC 正常情况下每个时钟节拍 +4，直到收到 IF_flush 信号后，下一时钟节拍，PC 值为跳转地址 JumpAddr 的值。仿真结果正确，IF 模块功能实现。

4.4 CPU 顶层模块仿真结果及分析

对于 CPU 顶层仿真操作有一些额外设置要求，将 Risc5CPU_tb.v 设为 Design Unit，将 Optimization Options 中的 Design Object Visibility 选中 Apply full visibility to all modules（full debug mode）。

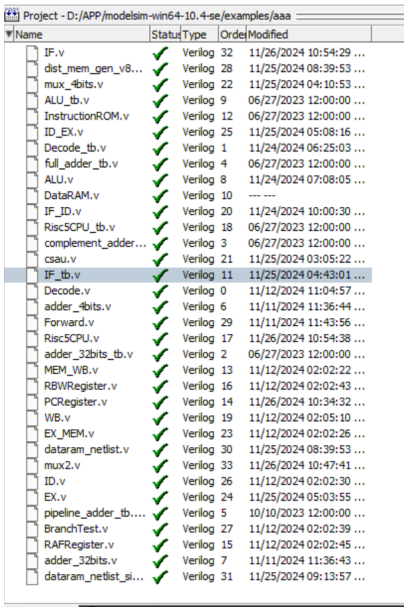


在 Libraries 中添加如下库：

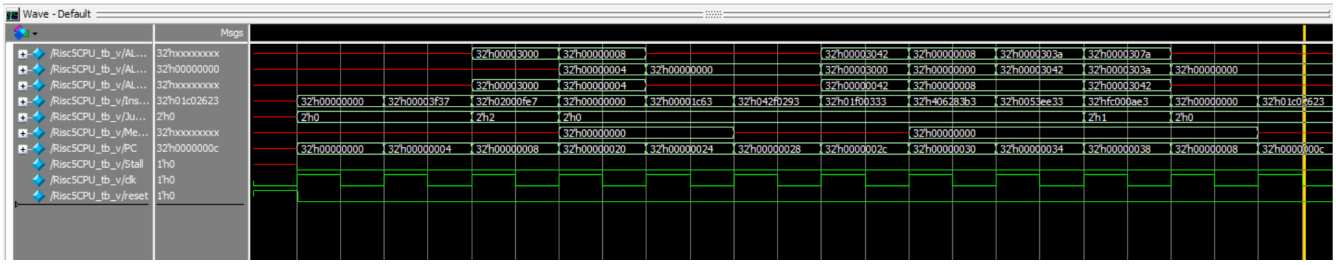


以及还需要在 Vivado 工程中找到 DataRAM.v ,dist_mem_gen_v8_0.v 和 DataRAM_sim_netlist.v，我这里的名字为 dataram_netlist.v,dataram_netlist_sim_netlist.v,dataram_netlist_sim_netlist.v, 并添加到 ModelSim 工程中，否则会报错 glbl 模块找不到。

Project 中的文件如下：



仿真结果如下：



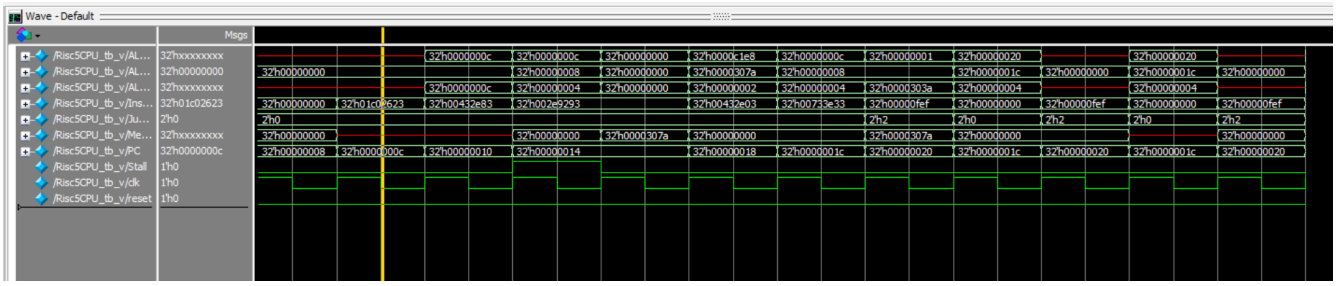


表 30.11　测试程序的运行结果

reset	clk	PC	Instruction（ID）	JumpFlag	Stall	ALU_A	ALU_B	ALUResult	MemDout（MEM）
1	1	0	0	0	0	0	0	0	-
0	2	4	00003f37 (lui)	0	0	-	-	-	-
	3	8	02000fe7 (jalr)	2	0	-	3000	3000	-
	4	20	0	0	0	4	4	8	-
	5	24	00001c63 (bne)	0	0	--	-	-	-
	6	28	042f0293 (addi)	0	0	-	-	-	-
	7	2c	01f00333 (add)	0	0	3000	42	3042	-
	8	30	406283b3 (sub)	0	0	0	8	8	-
	9	34	0053ee33 (or)	0	0	3042	8	303a	-
	10	38	fc000ae3 (beq)	1	0	303a	3042	307a	-
	11	8	0	0	0	-	-	-	-
	12	c	01c02623 (sw)	0	0	-	-	-	-
	13	10	00432e83 (lw)	0	0	0	c	c	-
	14	14	002e9293 (sll)	0	1	8	4	c	-
	15			0	0	-	-	-	307a
	16	18	00432e03 (lw)	0	0	307a	2	c1e8	-
	17	1c	00733e33 (sltu)	0	0	8	4	c	-
	18	20	0000fef (jal)	2	0	8	303a	1	307a
	19	1c	0	0	0	1c	4	20	-
	20	20	0000fef (jal)	2	0	-	-	-	-
	21	1c	0	0	0	1c	4	20	-

结合指令分析仿真波形：

lui X30, 0x3000：这条指令将立即数 3000 加载到 X30 的高 20 位。ALU_A 为 0，ALU_B 为 3000，ALUResult 为 3000，波形显示正确

jalr X31, later(X0)：跳转到 later 标签的地址，并将返回地址存储在 X31，JumpFlag 为 2，表示跳转，波形显示正确第 3 行，这里 PC 将更新为 jalr 指令的下一条指令地址，X31 存储返回地址，波形显示正确。

later: bne X0, X0, end：判断大小，如果寄存器 X0 的值不等于自身，则跳转到 end，由于条件不成立，PC 正常增加，波形显示正确。

addi X5, X30, 0x42：将寄存器 X30 的值加上立即数 0x42，结果存储在寄存器 X5 中，ALU_Result 显示加法结果 3042，波形显示正确。

add X6, X0, X31：将寄存器 X0（总是 0）和 X31 的值相加，结果存储在寄存器 X6 中，ALU_Result 显示加法结果 8，波形显示正确。

sub X7, X5, X6：将寄存器 X5 的值减去寄存器 X6 的值，结果存储在寄存器 X7 中，3042 - 8=303a，ALU_Result 显示减法结果 303a，波形显示正确。

or X28, X7, X5：将寄存器 X7 和 X5 的值进行逻辑或运算，结果存储在寄存器 X28 中，303a OR 3042=307a，ALU_Result 显示或运算结果 307a，波形显示正确。

beq X0, X0, earlier：如果寄存器 X0 的值等于自身（这总是成立），则跳转到 earlier 标签，JumpFlag 为 1，表示条件跳转，波形显示正确。

earlier：sw X28, 0x0C(X0)：定义一个标签 earlier，用于跳转和内存操作。将寄存器 X28 的值存储到内存地址 0x0C 处。ALU_Result 显示或运算结果 0x0C，波形显示正确。

lw X29, 4(X6)：从内存地址 X6+4 处加载一个字到寄存器 X29。

slli X5, X29, 2：将寄存器 X29 的值逻辑左移 2 位，结果存储在寄存器 X5 中，这里 X29 发生了数据冒险，因此 Stall=1，流水线阻塞，PC 指令不变，下一个时钟周期 Stall=0，流水线恢复正常，波形显示正确。

lw X28, 4(X6)：再次从内存地址 X6+4 处加载一个字到寄存器 X28，ALU__A = 307a，ALU_B 显示 offset4，ALUResult=cle8，波形显示正确。

sltu X28, X6, X7：如果寄存器 X6 的值小于寄存器 X7 的值，则将结果 1 存储在寄存器 X28 中，否则为 0，ALU_B 大于 ALU_A,ALU_Result 为 1，波形显示正确。

done:jal X31, done：定义一个标签 done，用于跳转。无条件跳转到 done 标签。JumpFlag=2, 波形显示正确。

接下来将进入无限循环，PC 不断重复 20 与 1c，不断跳转。

对照表 30.11，可以看出各项数据均正确，整体功能实现，设计成功。

5 遇到的问题及解决方法

（1）仿真时注意导入全部文件

仿真时一定要所有子模块代码导入到 ModelSim 的 project 中，不过大部分文件少了都会报错，看 error 就比较容易找出原因了，但是在顶层仿真时，以及还需要在 Vivado 工程中找到 DataRAM.v ，dist_mem_gen_v8_0.v 和 DataRAM_sim_netlist.v，并添加到 ModelSim 工程中，否则会报错 glbl 模块找不到，这个报错比较难找到原因，特此说明。

（2）要注意变量名的一致性：

在设计时，我把 IF 模块先赋值为 IF1，后面修改时又重新有了新的 IF 文件，顶层设计的时候又是 IF，导致新旧文件混杂，仿真结果有问题，同时我在 IF 文件中我设置了很多 PC 相关的命名代码，PC_IF,PC_reg,PC_instruction 等，同时有调用其他模块，其他模块可能只有 PC 变量，在输入输出选择时要注意名字。

（3）PC 跳转的问题

对于 IF 模块的设计，PC 跳转的问题困扰了我很久，到底该怎么设计，后面设计甚至出现了 IF 单独仿真成功但是 CPU 顶层仿真有问题，后面认真想了终于理解了：如果不发生跳转（无 Branch 或 Jump 信号），则直接从当前 PC 所指向的内存地址取指令，完成取指令操作后，PC+4；如果发生跳转，则需先修改 PC 值为 PC = JumpAddr，再从 PC 所指向的内存地址取指令，完成取指令操作后 PC+4，同时因为跳转的发生还需要清空当前流水线（因为无跳转时顺序取指令执行，跳转的发生使得预先取得的指令失效）；此外，如果发生阻塞（IFWrite=0），该时钟周期 PC 值需要保持不变。

（4）IP 核生成原因

还有一个解决了但未知原因的问题，就是我生成 DataRAM 的时候，完全按照指令操作，但是生成不了 IP 核（生成后的文件夹缺少文件且仿真失败），当时试了好几次，上网查了很多原因和解决方法都没有成功，我最后以为是我的 vivado 版本问题（我的是 vivado2023），但是后面重新换了个名字生成 IP 核就成功了，因此虽然解决了但还是没搞清楚具体原因。

（5）设计时可以多设置几个文件模块，增加代码清晰度

刚开始设计的时候，经常想把很多功能放在一个文件中完成，但是这样不仅已写的时候也经常会搞混，在报错时检查代码错误也很麻烦，后面我就开始尽量将不同功能分开写成模块，顶层设计时直接调用模块，不仅增加了清晰度便于他人阅读，自己修改也更方便了。

（6）= 和 <= 表示阻塞式赋值和非阻塞式赋值

在刚开始设计的时候，我经常疑惑应该用 assign 还是 always 块，后面入手设计慢慢理解了，如果描述的逻辑仅依赖于当前的输入值，并且输入变化时输出会立即变化，用 assign，用阻塞赋值 =；如果组合逻辑较为复杂，或者对于时钟触发的逻辑，或者需要使用控制流语句，就使用 always 块，并在块内大部分都使用非阻塞赋值 <=。如果只是该模块内部的信号变化，大部分

都为 assign，而如果和输入输出有关系，大部分为 always，同时调用模块时要注意 assign 还是 always，我之前有一个 PC 指令的调用内部应该是非阻塞赋值用了阻塞赋值，导致结果错误，找了好久才找出原因并理解。

6 思考题

如下面两条指令，条件分支指令试图读取上一条指令的目标寄存器，插入气泡或数据转发都无法解决流水线冲突问题。为什么在大多 CPU 架构中，都不去解决这一问题？这一问题应在什么层面中解决？

lw X28, 04(X6)

beq X28,x29,Loop

解决方案：

通过在 lw 和 beq 之间插入两个 NOP 指令，可以确保有足够的时间让 lw 指令完成其写回操作。这是因为在大多数处理器设计中，写回操作阶段通常发生在时钟周期的后半部分，而寄存器的读取操作通常在下一个时钟周期的前半部分进行。因此，插入两个 NOP 可以确保 beq 指令在 lw 指令的写回操作完成后才读取寄存器。这种方法简单且有效，但会增加执行指令所需的总时间。

在一些高频设计中，由于时钟周期非常短，即使插入两个 NOP 也可能不足以确保 lw 指令的写回操作完成。在这种情况下，插入三个 NOP 可以提供额外的时间，确保写回操作完成。

此外，插入一个 NOP 指令，并转发读取内存的值到 ALU 也同样可以解决该问题。

为什么不解决:

我认为这是性能与复杂性权衡后的结果，插入 NOP 是一种简单但可能导致性能损失的方法，有自己的开销和限制，特别是在高性能设计中，因为它增加了指令的执行时间；数据转发是一种更复杂的硬件解决方案，会增加硬件的复杂性和开销，可能导致性能下降。硬件解决方案不够高效，这个问题在别的层面解决开销更小。

问题应在什么层面中解决：

这一问题在编译器级别解决更好，编译器可以通过指令调度和重排来减少冒险发生的可能性；编译器还可以通过代码变换技术（如软件流水线）来优化代码，减少流水线冒险。